Corridor Games for Multi-Mobile-Sink Dubins-TSP

A critical review of the current game-theoretic conflict resolution, and a mathematically defensible redesign

Sources reviewed: `Dubins_TSP.pdf` (S. Dey, "Dubin's TSP in WSN", May 2026), Section 0.10 "Corridor Games" together with Sections 0.1–0.11; `corridor_game.py`; and the modules it depends on (`visualize.py`, `main.py`, `dubins.py`, `environment.py`) at repository commit 76233e8.

Date: 28 October 2026

> **How to read this document**
>
> Each main section opens with a short **"In short"** box in plain language. The text after it keeps every technical detail: definitions, proofs, assumptions and code locations.
>
> Code locations are written as *file, lines a–b* and refer to commit 76233e8.
>
> **Probe 1–5** are experiments run on the actual code. **Check [1]–[5]** are numerical checks of the proposed mathematics. Both are summarised in Appendix A.
>
> No repository files were modified while preparing this review.

Contents

Notation and key terms . . . 3

1. Executive verdict . . . 4

2. Approach 1: what the PDF says, and what is missing . . . 5

2.1 What Section 0.10 actually says . . . 5
2.2 Strengths . . . 5
2.3 Mathematical gaps . . . 6
2.4 Inconsistencies with the rest of the framework . . . 6

3. Approach 2: what corridor_game.py actually computes . . . 7

3.1 Line by line . . . 7
3.2 Is it game theory? . . . 7
3.3 Probe results on the actual code . . . 8
3.4 Termination, deadlock and safety . . . 8
3.5 Physical model and integration . . . 8
3.6 Is trajectory-list length a fair measure of urgency? . . . 9
3.7 Are duplicated positions enough? . . . 9
3.8 Implementation bugs versus conceptual problems . . . 9

4. Side-by-side comparison . . . 9

5. Recommended formulation . . . 10

5.1 Why this model . . . 11
5.2 Formal definition . . . 11
5.3 Feasible actions: generation and validation . . . 11
5.4 Conflict between two vehicles: continuous-time and conservative . . . 12
5.5 Individual cost and its link to lexmin(K, T_{max}, L_{total}) . . . 13

6. What can be proved, and under which assumptions . . . 13

6.1 Assumptions used below . . . 14
6.2 Consequences . . . 14

7. Algorithm (pseudocode) . . . 15
7.1 Top-level procedure . . . 15
7.2 Best-response dynamics and the cost . . . 16
7.3 Alternatives, conflict detection and helpers . . . 16
7.4 When no feasible alternative exists . . . 16
7.5 Guarantees . . . 16
8. Implementation architecture . . . 17
8.1 Data structures (corridor_game.py and trajectory.py) . . . 17
8.2 Functions . . . 17
8.3 Interfaces with existing modules . . . 18
8.4 Avoiding infinite loops . . . 18
8.5 Complexity . . . 18
8.6 Unit tests (pytest) . . . 18
9. Testing and evaluation plan . . . 19
9.1 Methods . . . 19
9.2 Instances . . . 19
9.3 Fair conditions . . . 20
9.4 Metrics . . . 20
9.5 Ablations . . . 20
9.6 Hypotheses (to be tested, not assumed) . . . 21
10. Research limitations and claims . . . 21
10.1 Standard results being used (cite, do not claim) . . . 21
10.2 Possibly novel (subject to a literature review) . . . 21
10.3 Claims that would be unjustified . . . 21
10.4 The honest caveat a reviewer will raise . . . 22
10.5 Other limitations . . . 22
Appendix A. Probes and numerical checks . . . 22
Appendix B. References . . . 23

Notation and key terms

Symbol	Meaning
MS, BS	Mobile sink; base station (also written S_0)
K, K_{max}	Number of deployed MSs; maximum number available
R	Minimum turning radius of an MS
v	Cruise speed. The MS either moves at speed v or is stopped: $v(t) \in \{0, v\}$
τ_i	Dwell (data-collection) time at sensor i
B	Mission-time budget of every MS
T_k, L_k	Completion time and path length of MS k
T_k^0, L_k^0	The same values for the nominal (pre-game) route
T_{max}, L_{total}	Makespan $\max_k T_k$; total length $\Sigma_k L_k$
ℓ, w	Vehicle length and width
ρ	Footprint (disc) radius: $\frac{1}{2}\sqrt{(\ell^2 + w^2)} + \delta_{margin}$
d_{sep}	Required centre-to-centre separation of two MSs: $2\rho + \delta_{margin}$
Δt	Step of the common time grid $t_n = n\Delta t$
N, N^c	Players (the MSs in one conflict cluster); all other MSs, whose trajectories stay fixed
A_k, a_k	Action set of MS k; one action. $a = (a_k)_{k \in N}$ is a joint profile; a_{-k} = actions of the others
J_k, Φ	Individual cost of MS k; potential function (global scoreboard)
χ_{kj}	Conflict indicator of MSs k and j: 1 if they come too close at some time, else 0
NE, FIP, GNEP	Nash equilibrium; finite improvement property; generalized Nash equilibrium problem

Key game-theory terms in plain words

- **Game:** players, the actions each can take, and a cost for each player that depends on *everyone's* actions.
- **Best response:** the action that gives a player its lowest cost, given what the others are doing now.
- **Pure Nash equilibrium (NE):** one action per player such that no single player can lower its own cost by changing only its own action.
- **Exact potential game:** a game with one global scoreboard Φ. Whenever one player changes action, its own cost changes by exactly the same amount as Φ. So every selfish improvement also lowers Φ.
- **Finite improvement property (FIP):** any sequence of one-player-at-a-time strict improvements must stop after finitely many steps.
- **Lexicographic comparison:** compare the first component; only if it is equal, compare the second; and so on (like dictionary order).

1. Executive verdict

> **In short**
>
> Neither existing approach is a properly defined game, and neither should be kept unchanged. The PDF sketches the outline of a game but leaves out the costs, the interaction between players, time, and what the strategies mean as real paths. The Python code is a priority rule that makes one robot pause; it never computes payoffs, best responses or equilibria.
>
> The recommended replacement is an **exact potential game**. Each mobile sink chooses from a short list of complete, pre-checked tour variants (route + waiting schedule). Safety is certified by a separate validation step, not by the equilibrium.

Approach 1 — PDF Section 0.10

- Names players, strategy labels {Left, Right, Detour} and best-response updates.
- Does **not** define a cost function, how one player's choice affects another, time, or the trajectory behind each label. So one cannot claim that a pure NE exists or that best response converges.
- Algorithm 1 restores a player's old route after convergence if the budget is violated. This destroys any equilibrium that was reached.
- In the current code, any corridor that passes the PDF's "narrow" test is closed to the planner, so users(c) = ∅ always (Section 2.4).

Approach 2 — corridor_game.py

- A priority scheduler that looks one frame ahead. Priority = length of the trajectory list + 10 × consecutive waits.
- No payoffs, best responses or equilibria are computed.
- Read as a two-player "Chicken" game, both (Go, Yield) and (Yield, Go) are pure NE *whatever the scores are*. So calling the outcome an "equilibrium" adds nothing to the priority rule.
- With three or more robots, pairwise decisions overwrite each other.
- Probes on the real code: a head-on encounter is "resolved" with the robots still **0.000 m** apart; a pass at true distance **1.70 m** is never detected; the frame index is not physical time.

Recommendation

- Exact potential game over **pre-validated route-and-schedule alternatives**.
- Pairwise conflict costs that are the same for both vehicles of a pair (symmetric).
- Lexicographic individual cost aligned with lexmin(K, T_{max}, L_{total}).
- One-player-at-a-time strict best response — provably stops at a pure NE.
- Exact search over the small cluster if the equilibrium still contains a conflict.
- Independent continuous-time validation of the final plan. **Safety comes from this validation, not from the equilibrium.**

2. Approach 1: what the PDF says, and what is missing

> **In short**
>
> The overall idea is sound: settle conflicts locally after routing, let only the involved sinks choose, reject choices that break the budget, and use best-response updates. But the PDF never says what each sink pays, how sinks affect each other, or what “Left”, “Right” and “Detour” are as actual paths. Without these there is no game to analyse. It also contradicts itself on the budget rule and does not fit the current code.

2.1 What Section 0.10 actually says

1. **Narrow corridor:** a corridor $c \in C$ is narrow if $w_c < 2(R + \text{clearance})$.
2. **Corridor users:** users(c) = {k | Γ_k traverses c}.
3. **Congestion:** if |users(c)| > capacity(c), the corridor is congested and the game starts.
4. **Players and strategies:** the users of the congested corridor are the players. Every player gets the same set S_c = {Left, Right, Detour}.
5. **Game step:** each MS “evaluates the cost/latency associated with each feasible strategy” and picks the minimum.
6. **Budget:** Step 6 says a strategy giving $T_k > B$ is rejected and the previous route is kept. **Algorithm 1** instead checks this *after* the best-response loop and restores the previous route.
7. **Best response:** MSs update one at a time “until no MS can improve its own cost”.
8. **Output:** the updated routes Γ′.

Section 0.11 calls the game in Steps 8–10. Step 11 re-checks Dubins feasibility and $T_i \le B$. Step 12 says “re-optimize the affected routes and repeat”.

2.2 Strengths

- Running conflict resolution **after routing**, with only the conflicting MSs as players, is a sound way to split the problem.
- Rejecting strategies that break the budget is the right instinct.
- One-at-a-time best response is the correct update rule *for potential games*.
- The narrowness test has a useful physical meaning that the PDF does not state. If clearance = footprint radius + safety margin, then $w_c < 2(R + \text{clearance})$ is exactly the condition under which a forward-only Dubins vehicle **cannot make a U-turn inside the corridor** (a U-turn needs a strip 2R + 2·clearance wide). In plain words: once an MS enters, it cannot turn back, so a head-on meeting inside cannot be fixed. Conflicts must therefore be settled *before* entry. This is worth saying explicitly in the paper.

2.3 Mathematical gaps

Item	What the PDF says	What is needed for rigour
Corridor set C, width w_c	Assumed given	A method to extract corridors from the obstacles; whether w_c is measured on original or inflated obstacles.
capacity(c)	Not defined	Lanes? Vehicles at once? Flow per unit time? It must depend on direction: one lane holds any number of same-direction vehicles at headway $\geq d_{sep}$, but **zero** opposing pairs.
Congestion test	Static count of users	Conflicts depend on **space and time**. Two MSs passing hours apart do not conflict (false positive). Conflicts at corridor mouths or in open space are missed (false negative).
Strategy set	Same three labels for everyone	Each label must map to a concrete timed trajectory. "Detour" is not unique and may use another corridor. There is **no waiting/timing strategy**, so a head-on conflict in a single lane with no detour cannot be resolved inside the strategy set.
Cost J_k	"cost/latency", no formula	A function $J_k(a_k, a_{-k})$. If J_k depends only on a_k, each MS solves its own separate problem; the "equilibrium" is just each MS's individual best choice — likely the same lane for all — and the conflict remains.
Game model	Players and labels only	With no cost, no normal-form game (N, S, J) exists. No extensive-form or Stackelberg structure is defined either.
Pure NE existence	Assumed	Finite games need not have one (e.g. matching-pennies-type costs). Existence needs structure, e.g. a potential.
Convergence	"Repeat until no MS can improve"	Best-response updates can cycle forever in finite games without a potential. Convergence needs the FIP.
Budget rule	Step 6 and Algorithm 1 disagree	Restoring a route after convergence (a) returns a profile that is not the converged one, so it is not an NE; (b) usually brings the original conflict back. If latency counts toward T_k, then T_k depends on a_{-k}, the constraint is *coupled*, and the problem becomes a GNEP. Not addressed.
Several corridors	foreach c in sequence, no re-check	A Detour chosen in game c can congest a corridor c′ that was already processed. The result depends on the processing order and is not a fixed point.
Deadlock	Not addressed	—
Dubins feasibility	Not addressed	A lane shift by lateral offset h needs an S-manoeuvre (two opposite arcs of radius R) of length $\sqrt{(h(4R - h))}$ (for $h \leq 2R$) in free space **before** entry, plus heading aligned with the corridor at entry. Example: R = 5, h = 2 needs 6.0 m. Under the PDF's own test, a Detour is impossible once inside.
Obstacles and separation after changes	Algorithm 1 re-checks only $T_k \leq B$	Constraints (19) (obstacles) and (20) (separation) must be re-validated; Section 0.11 Step 11 does not list them.
Termination of Section 0.11	Step 12 says "repeat"	No bound. A bounded escalation ladder is needed.

2.4 Inconsistencies with the rest of the framework

- **Constraint (20) cannot hold at t = 0.** It applies for all $t \in [0, \min(T_k, T_l)]$, but every MS starts at S_0 at t = 0. Fix: staggered departures or an explicit docking exemption at the BS. The code silently uses a 5-unit exemption (*corridor_game.py, line 47*).
- **Stop-and-go is implied but not stated.** Sections 0.1 and 0.3 say "constant velocity v", but Section 0.4 has stationary dwell times τ_i. The real model is $v(t) \in \{0, v\}$. State it, because waiting is only kinematically allowed if stopping is.
- **Eq. (5) understates the footprint.** $r_{safe} = w/2 + \delta$ covers a rotating rectangle only if checks use the oriented footprint. For a disc model use $\rho = \frac{1}{2}\sqrt{(\ell^2 + w^2)}$.

- **Narrow corridors cannot occur in the current code:**
  - *main.py, lines 167–173* accept an obstacle only if its (d_{safe} + 2R) buffer does not overlap another obstacle's buffer. So physical gaps are always **> 2(d_{safe} + 2R) = 24**.
  - *visualize.py, line 19* inflates obstacles by 12 per side, which closes any gap under 24 to the planner.
  - The PDF's narrowness threshold is 2(R + clearance) = 14 (with clearance = d_{safe} = 2).
  - So no planned route ever traverses a "narrow corridor", and users(c) = ∅ always (Probe 5). The conflicts the code actually meets happen in open space.

3. Approach 2: what corridor_game.py actually computes

> **In short**
>
> Frame by frame, the code checks whether two robots are closer than 3 units *at the same list index*. If so, the robot with the smaller score (list length plus a waiting bonus) pauses for one frame, implemented by copying its previous position. That is a priority rule, not a game. It also has bugs that let collisions through.

3.1 Line by line

Lines	What happens
18–95	Loop over the frame index n, starting at 1. Line 22: a robot is active while n < len(traj).
29	Every active robot starts with "Go".
31–47	For each pair, compare the positions at the *same index* n. Conflict if $\|p_k[n] - p_j[n]\| < 3.0$ and the two are not both within 5 units of the BS.
54–68	Score $w_k = \text{len}(traj_k) + 10 \cdot wait_k$, or ∞ if $wait_k > 50$.
73–82	Lower w → Yield; on a tie the lower index yields. Decisions go into one shared dict, so **later pairs overwrite earlier ones**.
85–93	Yield: insert traj[n−1] at index n, which delays the whole rest of the trajectory by one frame. Go: reset that robot's wait counter.
95	n += 1. The modified frame n is never checked again.

3.2 Is it game theory?

Element	Present?	Evidence
Players	Only implicitly: whichever pairs are close at frame n	lines 38–46
Action set	{Go, Yield}, one frame at a time	lines 29, 74–82
Utility / cost	**No.** w_k is a priority key. It is never compared with a cost of going, colliding or both waiting	lines 54–68
Strategic dependence	**No.** No decision considers what the other robot would do	—
Best responses	**Never computed**	—
Nash equilibrium	**Never computed or checked**	—

Reading it as a game (Chicken). Suppose collision costs C, yielding costs w_k < C, and going costs 0. The best response to Go is Yield, and the best response to Yield is Go. So **both** (Go, Yield) and (Yield, Go) are pure NE for any w. Choosing the one where the lower-w robot yields is only *equilibrium selection by social cost*, which is exactly the priority rule. The NE label adds no information. With three or more robots, the overwrite in lines 73–82 creates profiles that are not equilibria of any game: in Probe 3b, a pair 2.0 m apart are both told "Go", and in any game that penalises collisions each would rather Yield.

Verdict: a priority-based, frame-local scheduling heuristic. It is the legacy `collision_resolution.py` with a different sort key.

3.3 Probe results on the actual code

Probe	Setup	Result
1. Head-on in a line	Two robots, 2.5 m per frame, opposite directions	One wait inserted. **Minimum distance after "resolution": 0.000** — the Go robot drives through the stopped one.
2. Opposing pass, lateral offset 1.7	Sampled closest distance 3.023	**Not detected.** True continuous minimum is **1.701**, below both `safe_dist = 3.0` and `env.d_safe = 2.0`. Trajectories unchanged.
3 / 3b. Three robots in a row	Pairs (0,1) and (1,2) conflict	Depending on scores: either everyone yields needlessly, or **the conflicting pair (0,1) are both told Go**.
4. Frame vs time	Real 3-sensor tour from `generate_interpolated_trajectory`	17 steps are not 2.5 m long. Frame count implies **35.00 s**; true time is **32.78 s** (+2.22 s). Causes: each segment is interpolated separately (*visualize.py, lines 62–86*), segment end points are appended, junction points are duplicated, and dwell is truncated (*visualize.py, line 111*). The drift differs per MS, so **frame n of MS A and frame n of MS B are different physical times**.
5. Do corridors exist?	Obstacle generator and planner inflation	Gaps are always > 24 while the narrow threshold is 14 (Section 2.4).

3.4 Termination, deadlock and safety

- **It does terminate — but for the wrong reason.**
  - In every frame, robots with no conflict are Go, and the last pair processed always has one Go. So at least one active robot advances.
  - Advancing reduces that robot's remaining frames by 1; yielding leaves them unchanged. So Σ_k (len_k − n) falls by at least 1 per frame, which bounds the number of iterations by the total initial length.
  - This works only because **a Go robot keeps moving into the conflict**. Robots are points that can pass through each other.
- **Deadlock** is avoided the same way — by ignoring physics.
- **Safety is not ensured**, for five reasons:
  - (1) the state actually executed at frame n (the yielder at its old position, the other at its new one) is never checked;
  - (2) the overwrite can leave a conflicting pair both on Go;
  - (3) conflicts between samples are missed;
  - (4) the index is treated as time;
  - (5) finished robots are dropped from the check (lines 22, 33, 40) and padded only afterwards (*visualize.py, line 232*).

3.5 Physical model and integration

- **Heading and turning radius:** ignored; only positions are used. A wait is a stop. That is feasible only under the stop-and-go assumption, and the stop may be inside a corridor or in another MS's path.
- **Obstacles:** geometry is not changed, so the game adds no obstacle violations. But the nominal paths already contain unsafe fall-backs: a straight line when Dubins fails (*dubins.py, lines 240–241*) and a straight line through obstacles when A* fails (*visualize.py, line 59*).
- **Mission budget:** `env.budget` is never referenced, although the docstring says it is.
  - The budget is checked in `main.py` *before* visualization, using **Euclidean** length (*main.py, lines 6–24*).
  - The game runs only inside the visualizer (*visualize.py, lines 224–225*).
  - So the inserted waits are **never budget-checked**.
  - Steps 8–10 in `main.py` do nothing (`updated_routes = routes`, *main.py, line 81*).
- **Continuous motion:** not modelled.

3.6 Is trajectory-list length a fair measure of urgency?

Only as a crude proxy. With a common budget B, a larger total T_k means less slack $B - T_k$, so "longer list ⇒ more urgent" points the right way. But:

- it counts frames, which include interpolation artefacts (Probe 4);
- it grows by 1 with every wait, which reinforces itself;
- the +10 per wait and the cap at 50 are arbitrary and mix units;
- it ignores what yielding costs the other robot, and conflicts that yielding creates later.

The principled replacement is **slack against the makespan reference**, which appears in Section 5.5 as E_k.

3.7 Are duplicated positions enough?

No. Inserting a duplicate is a valid open-loop stop only if **all** of the following hold, and none is checked:

- stopping is allowed by the kinematic model;
- the stop location is clear of other robots' swept paths for the whole wait;
- the rest of the trajectory, now shifted in time, is re-checked in continuous time against everyone;
- the budget still holds;
- instantaneous stop/start is declared as a modelling assumption.

For a vehicle that truly moves at constant speed (e.g. a fixed-wing UAV), waiting must instead be a loiter circle.

3.8 Implementation bugs versus conceptual problems

Implementation bugs	Conceptual problems
• Pairwise decisions overwrite each other (lines 73–82) • No re-check after a frame is modified (lines 85–95) • Frame index used as time (visualizer pipeline) • `safe_dist = 3.0` hard-coded, unrelated to `env.d_safe = 2.0` • Finished robots ignored • Input lists mutated in place • `list.insert` makes the worst case $O(F^2)$ • Budget never used • Called after the budget check	• No payoffs and no strategic interaction • One-frame, myopic decisions • No detour or timing alternatives • No equilibrium object • Safety inferred from sampled positions • "Urgency" not tied to the objective

4. Side-by-side comparison

> **In short**
>
> Both approaches miss the two things that make something a game: individual costs and interaction between players. Neither handles time, Dubins feasibility or multi-robot conflicts soundly. Neither should be kept as it is.

Criterion	Approach 1: PDF	Approach 2: Python	Technical assessment
Player definition	Users of a congested corridor	Pairs close at frame n, recomputed every frame	PDF acceptable at cluster level; Python has no persistent players
Strategy / action set	{Left, Right, Detour}, labels only	{Go, Yield} per frame	Neither maps to validated trajectories. PDF has no timing action; Python has no route action
Individual cost	Not defined	None (only a priority key)	**Both missing**

Criterion	Approach 1: PDF	Approach 2: Python	Technical assessment
Interaction between players	Not defined	None evaluated	**Both missing.** Without coupling there is no game
Congestion modelling	Static |users| > capacity; capacity undefined	None	Neither models time or direction
Conflict detection	Spatial overlap with a corridor	Same-index Euclidean check, threshold 3.0	PDF ignores time; Python's index ≠ time and it misses conflicts between samples
Feasibility constraints	Budget only, applied inconsistently	None	Both incomplete
Best-response dynamics	Named	Absent	PDF has the right concept for potential games
NE definition	Implied, not defined	Claimed in comments only	Neither defines one
NE existence	Not established	Vacuous (any Go/Yield split of a pair)	—
Convergence	Assumed	Terminates, but by letting robots pass through each other	Neither has a valid convergence argument for conflict *resolution*
Deadlock handling	None	Force Go after 50 waits; robots tunnel through	Neither
Multi-player conflicts	Corridors processed in sequence, no re-check	Pairwise decisions overwrite each other	Both unsound
Dubins feasibility	Not checked	Not considered	Neither
Obstacle avoidance after changes	Not re-checked	Geometry unchanged (inherits planner fall-backs)	—
Mission-time constraint	Rejection after convergence breaks the NE	Ignored; waits never budget-checked	Both inadequate
Inter-vehicle collision avoidance	Not validated	Not ensured (Probes 1–3)	Both fail
Computational complexity	Undefined	$O(F \cdot K^2)$ checks + $O(F^2)$ inserts	—
Fit with the full framework	Concept only; with current inflation, corridors never qualify	Runs in the visualizer, after budget verification	Neither is integrated correctly

Conclusion: keep neither unchanged. From the PDF, keep the cluster-level players, budget filtering, best-response dynamics, and the "no U-turn inside" reading of $w_c < 2(R + \text{clearance})$. From the code, keep no algorithm — only the hypothesis that holding resolves most open-space encounters, which becomes the "hold-only" ablation in Section 9.

5. Recommended formulation

> **In short**
>
> Each MS gets a short **menu** of complete, already-checked tour variants — for example "as planned", "wait 6 s longer at sensor S17", or "use the other gap around obstacle 4". Each MS picks one item from its own menu.
>
> Its cost counts, in order of importance: (1) how many other MSs it would come too close to; (2) how far it would push the fleet's finishing time past the current worst; (3) extra distance; (4) waiting time.
>
> Because a near-miss between A and B costs A and B the same, the whole system has one scoreboard Φ. Every selfish improvement by one MS lowers Φ, so the process cannot go round in circles.

5.1 Why this model

Model	Verdict	Reason
Rosenthal congestion game over corridor resources	Rejected as the primary model	Its potential needs latencies $\ell_e(n_e)$ that depend only on the **number** of users and are the same for every player. Corridor delay depends on arrival order, direction and timing, and spatial resources do not encode time. Different vehicle sizes give weighted congestion games, which guarantee pure NE only for affine or exponential latencies.
Generalized potential game	Avoided by design	Arises if delays caused by others count toward $T_k \le B$, or if "no conflict" is a hard joint constraint. Workable with a shared constraint set, but it needs a feasible starting profile — and finding a conflict-free feasible profile is the very problem being solved.
Finite exact potential game with symmetric pairwise conflict costs; actions = pre-validated route-and-schedule alternatives	**Chosen**	Waiting is part of a player's **own action** (chosen, not imposed by others). So T_k depends only on a_k, the budget does not couple players, and the joint action space is the plain product ΠA_k.

5.2 Formal definition

$$G = (N, (A_k)_{k\in N}, (J_k)_{k\in N})$$

- **N** — one *conflict cluster*: a connected component of the graph whose edges are pairs of MSs whose nominal trajectories conflict. The other MSs $j \in N^c$ keep fixed trajectories (written with actions $\bar{a}_j$).
- **A_k** — a finite set of actions $a = (r, h, \delta)$:
  - $r \in R_k$ is a route variant;
  - $h \in H_k(r)$ is a hold location;
  - $\delta \in \Delta_k$ is a hold duration.
- Each action defines a fully timed trajectory $\Gamma_k^a : [0, T_k(a)] \to SE(2)$. An action is in A_k only if it passes the individual validation of Section 5.3. The nominal action $a_k^0 = (r_0, –, 0)$ is always included if it validates.
- **a** $= (a_k)_{k\in N}$ is a joint profile; a_{-k} denotes everyone else's actions.

Choice of action representation: trajectory-and-schedule alternatives. The other options are not enough:

- choosing a *corridor resource* has no timing and is not a trajectory;
- choosing a *route only* cannot resolve a head-on conflict in a single lane without a detour;
- choosing a *time slot only* cannot express detours, and a reservation still needs a trajectory behind it.

Corridors and time windows become *attributes* of an alternative — used to generate and prune alternatives — not game resources.

5.3 Feasible actions: generation and validation

Generation (sensor assignment and visiting order stay fixed, so K and coverage are untouched)

1. **Nominal:** the current tour with the existing Dubins smoothing.
2. **Holds:** hold only where the MS is *already stationary* under the dwell model: the last sensor visited before its earliest conflict time t_k^c (extend $\tau_i \to \tau_i + \delta$), or a departure delay at the BS (which also implements staggered departures). This adds no kinematic assumption beyond the dwell model. Values of δ:
   - a grid $\{0, \Delta_h, 2\Delta_h, \ldots\}$ up to the slack $B - T_k(r)$;
   - plus *targeted* values from the nominal conflict records, e.g. $\delta^* = t_{exit,j} - t_{entry,k} + (d_{sep} + v\Delta t)/v$ ("let j clear first");

- if the platform cannot stop: loiter circles of radius ≥ R in a validated free disc, with $\delta \in \{n \cdot 2\pi R/v\}$.

3. **Route variants:** for the leg (waypoint i → i+1) that contains the conflict, re-run the visibility-graph shortest path with the edges crossing the conflict region removed. This gives a different homotopy class. Then apply the same heading assignment and Dubins smoothing. **Lane variants** (lateral offset $\pm d_{sep}/2$) are offered only if the corridor's free width is $\geq d_{sep}$ after inflating by $\rho + \delta_{margin}$, and only if the $\sqrt{(h(4R - h))}$ lane change fits before entry. That fit is enforced by checks V1 and V2, not assumed.
4. **Combine and prune:** combine r × h × δ and cap at A_{max} (about 20–40), keeping alternatives that are Pareto-nondominated in (E, ΔL, W) plus some spread in timing.

Validation V(Γ) — per MS, independent of the other players

Check	Rule
V1 Curvature / forward-only	Every segment is an L, S or R primitive with radius ≥ R. Heading is continuous at every junction ($\lvert\Delta\theta\rvert \leq \varepsilon_\theta$). Speed ∈ {0, v}. **Reject** the straight-line fall-backs in *dubins.py, line 241* and *visualize.py, line 59*.
V2 Obstacles	Footprint disc of radius $\rho = \tfrac{1}{2}\sqrt{(\ell^2 + w^2)} + \delta_{margin}$. Sample at arc-length spacing s and require dist(sample, original obstacles) $\geq \rho + s/2 + e_{sag}$, where $e_{sag} = R(1 - \cos(\Delta\varphi/2))$ is the chord sag. This conservatively guarantees clearance along the continuous path. Check against the *original* polygons, not the 2R-inflated ones.
V3 Budget	$T_k(a) = L_k(r)/v + \Sigma\, \tau_i + \delta \leq B$, using the **actual** trajectory length (not the Euclidean length of *main.py, line 24*).
V4 Return	The trajectory ends at S_0 and the tour is complete.

A hold shifts the rest of the trajectory later in time. This is handled automatically, because every alternative is a fully re-timed trajectory that is checked against everyone else's full trajectory.

5.4 Conflict between two vehicles: continuous-time and conservative

All trajectories share a common time grid $t_n = n\Delta t$. Positions $p_k^{a}(t_n)$ are computed **exactly** from the arc-length parameterisation, not from a frame index.

BS policy (explicit modelling assumption, replacing the code's 5-unit rule): an MS waits at a dock of radius r_{dock} before departure and after finishing; docked MSs are exempt from constraint (20).

$$d_{kj}(a_k, a_j) = \min_n \lVert p_k^{a_k}(t_n) - p_j^{a_j}(t_n) \rVert \text{ (over times when neither MS is docked)}$$

$$\chi_{kj}(a_k, a_j) = 1 \text{ if } d_{kj} < d_{sep} + v\Delta t + \varepsilon_{geo}, \text{ else } 0, \text{ with } d_{sep} = 2\rho + \delta_{margin}$$

Symmetry $\chi_{kj}(a_k, a_j) = \chi_{jk}(a_j, a_k)$ holds by construction, because distance is symmetric.

Lemma (conservative sampling)

If both vehicles move at speed ≤ v and $d(t_n) \geq d_{sep} + v\Delta t$ at every grid point, then $\lVert \Gamma_k(t) - \Gamma_j(t) \rVert \geq d_{sep}$ for **all** times t.

Proof. Any time t lies within Δt/2 of a grid point t*. In that time each vehicle moves at most vΔt/2, so the relative position changes by at most vΔt. By the triangle inequality, $d(t) \geq d(t^*) - v\Delta t \geq d_{sep}$. ∎

Check [5]: 2,000 random speed-bounded motion pairs, 0 violations. With Δt = 0.1 s and v = 5, the safety margin is 0.5 m.

5.5 Individual cost and its link to lexmin(K, T_{max}, L_{total})

Quantity	Definition	Unit
$C_k(a)$	$\Sigma_{j\in N,\, j\neq k}\, \chi_{kj}(a_k, a_j) + \Sigma_{j\in N^c}\, \chi_{kj}(a_k, \bar{a}_j)$ — conflicts with players and with fixed non-players	count
$E_k(a_k)$	$\max(0, T_k(a_k) - T_{ref})$, where T_{ref} = max over **all** MSs of the pre-game T_j^0	s
$\Delta L_k(a_k)$	$L_k(a_k) - L_k^0$	m
$W_k(a_k)$	total hold time δ	s

$$J_k(a) = (\, C_k(a),\, E_k(a_k),\, \Delta L_k(a_k),\, W_k(a_k)\,) \text{ compared lexicographically}$$

How each term relates to the global objective

- **K:** the game never changes the assignment, so K is fixed. Changing K is the outer loop's job.
- **C first:** constraint (20) is a hard constraint of the original problem; feasibility comes before optimality.
- **E second:** $\Sigma_k E_k = 0 \Leftrightarrow T_{max} \leq T_{ref}$, and $\Sigma_k E_k \geq T_{max} - T_{ref}$. It is an *upper-bound proxy* for priority 2, not T_{max} itself. Side effect: MSs that are not on the critical path can hold for free within their slack $T_{ref} - T_k^0$ — the principled replacement for list length.
- **ΔL third:** $\Sigma_k \Delta L_k = L_{total} - L_{total}^0$ exactly (priority 3).
- **W last:** waiting is not in the global objective, so it is only a tie-breaker. If data latency matters, put it into the global objective explicitly; the local game should not invent its own priorities.
- **Budget B:** a hard constraint, enforced through A_k.

Why the suggested $\alpha T_k + \beta \Delta L_k + \gamma D_k + \eta C_k$ was refined

- $T_k = T_k^0 + \Delta L_k / v + W_k$, so T_k, ΔL_k and the delay D_k are linearly dependent and double-count.
- Mixing seconds and metres needs arbitrary scaling.
- If "congestion delay" is *caused by others*, T_k depends on a_{-k} and the budget becomes coupled.
- So congestion should enter as a symmetric conflict count, not as a latency.

If a single number is needed

$$J_k = M \cdot C_k + w_E \cdot E_k + (w_L / v) \cdot \Delta L_k + w_W \cdot W_k$$

On the finite set A this reproduces the lexicographic order if $M > w_E \cdot \text{span}(\Sigma E) + w_L \cdot \text{span}(\Sigma \Delta L)/v + w_W \cdot \text{span}(\Sigma W)$, where $\text{span}(\cdot) = \max - \min$ over A; likewise $w_E \cdot \min^+ \Delta(\Sigma E)$ exceeds the spans of the terms below it, where $\min^+ \Delta(\cdot)$ is the smallest non-zero difference over A; and so on down the list.
Recommendation: implement tuples instead — no weights to tune and no scaling argument to defend.

6. What can be proved, and under which assumptions

> **In short**
>
> The cost design makes the game an **exact potential game**. That gives three results: a pure NE always exists; one-at-a-time strict improvements always stop; and they stop at an NE.
>
> But an NE is only a *local* optimum and can still contain a conflict. So an exact search over the cluster and an independent safety check are still needed.

Theorem 1 (exact potential)

Define $\Phi(a) = (\Sigma_{k<j\in N} \chi_{kj}(a_k, a_j) + \Sigma_{k\in N} \Sigma_{j\in N^c} \chi_{kj}(a_k, \bar{a}_j), \Sigma_k E_k, \Sigma_k \Delta L_k, \Sigma_k W_k)$.

Then for every player k, every profile a and every alternative $a'_k \in A_k$, component by component:

$J_k(a'_k, a_{-k}) - J_k(a_k, a_{-k}) = \Phi(a'_k, a_{-k}) - \Phi(a_k, a_{-k})$.

Proof. When only k changes, every term not involving k cancels. Pair terms (j, k) with $j < k$ equal χ_{kj} by symmetry, so the change in Φ's pair sum is $\Sigma_{j\neq k} [\chi_{kj}(a'_k, a_j) - \chi_{kj}(a_k, a_j)]$. The background, E, ΔL and W terms depend only on a_k. ∎

- Lexicographic order on $\mathbb{R}^4$ is a total order that is preserved under translation. So J_k strictly decreases lexicographically **if and only if** Φ does. The same holds for any positive-weighted scalarisation.
- **If χ were asymmetric** (e.g. conflicts charged only to the follower), the identity fails and one **cannot** claim an exact potential.
- Check [1]: on 200 random games, testing every unilateral deviation, the largest $|\Delta J_k - \Delta\Phi|$ was 8.9×10^{-16}.

6.1 Assumptions used below

ID	Assumption
A1	Each A_k is finite and fixed during the dynamics (or enlarged only finitely often, keeping the current action available).
A2	One player updates at a time.
A3	A player switches only on strict improvement (tolerance ε on the continuous components).
A4	Costs are deterministic and cached.
A5	χ is symmetric.
A6	E, ΔL, W and the budget depend on the player's own action only (decoupled constraints).

6.2 Consequences

1. **Existence.** If every $A_k \neq \emptyset$, Φ attains a lexicographic minimum on the finite set ΠA_k, and that minimiser is a pure NE.
2. **Termination (FIP).** Under A1–A6, Φ strictly decreases in a total order on a finite set. So every improvement path is finite and ends at a pure NE. **A finite action set alone is not enough: without A2, simultaneous updates can cycle** — Check [4]: (0,0) → (1,1) → (0,0) → …, exactly the "both wait, both go" livelock.
3. **Iteration bound.** Finite but exponential in the worst case. Computing pure NE is PLS-complete for congestion games, so no polynomial guarantee exists. In random tests, steps averaged 2.9 with a maximum of 9 (Check [2]). Still impose a cap; if it is hit, **report "not converged" and do not claim an NE**.
4. **Ties and cycling** are impossible under A2–A4, because Φ strictly decreases. Detecting a repeated state is only a bug detector (it catches an asymmetric χ or non-deterministic evaluation).
5. **An NE is not the same as conflict-free.** An NE is only a *local* minimum of Φ against single-player deviations.
   - Counterexample (Check [3]): $\chi(0,0) = \chi(0,1) = \chi(1,0) = 1$ and $\chi(1,1) = 0$. Profile (0,0) is a pure NE *with* a conflict, while (1,1) is conflict-free. Such structures arise at junctions where the fix needs two MSs to change at once.
   - So if $C(a^*) > 0$ after best response, run an **exact lexicographic minimisation of Φ** over the cluster. A global minimiser is also an NE, and it is conflict-free if and only if some conflict-free profile exists in

ΠA_k.

6. **Mission time** enters by filtering A_k (A6), so the argument holds unchanged.
   - The alternative design — delays caused by others — couples the constraints. A shared feasible set $X = \{a : T_k(a) \le B\ \forall k\}$ arises. Restricting deviations to stay inside X keeps the descent and stops at a *generalized* NE, but it needs a feasible starting profile in X, and the equilibrium becomes "socially constrained". The recommended design avoids this.
7. **A potential from corridor latencies** is valid only in Rosenthal form, $\Phi = \Sigma_e \Sigma_{m=1}^{n_e} \ell_e(m)$: anonymous latencies that are the same for every player, unweighted players, costs additive over resources. Own-action-only terms $d_k(a_k)$ can be added without losing exactness (this is how Approach 1 is formalised for the experiments). Player-specific latencies are not a potential game in general (Milchtaich); weighted players need affine or exponential latencies (Harks & Klimm); FIFO queueing delays do not fit the form at all.
8. **Efficiency.** With respect to Φ the price of stability is 1, because the global minimiser is an NE. **No bound on T_{max} is claimed.**
9. **Safety is not a property of the equilibrium.** It follows from three things together:
   - $\chi = 0$ for all pairs, under the sampling lemma;
   - V1–V4 passing for every trajectory;
   - an independent joint validation of the final plan.

7. Algorithm (pseudocode)

> **In short**
>
> Build exact timed trajectories, find conflicting pairs, and group them into clusters. For each cluster, generate menus of validated alternatives, let the players improve one at a time, and fall back to an exact search if a conflict remains. If a solved cluster creates new conflicts with others, merge the clusters and solve again. Finally, validate the whole plan independently.

7.1 Top-level procedure

```
procedure RESOLVE_CONFLICTS(env, routes, B, P)
  # P: Δt, ρ, d_sep, r_dock, Δ_h, A_max, iter_cap, exact_limit, aug_rounds
  1  for k in routes:  Γ[k] ← BUILD_TIMED_TRAJ(env, routes[k], variant=r0, holds=∅, Δt)
  2      if not VALIDATE_INDIVIDUAL(env, Γ[k], B):  return (Γ, report, NOMINAL_INFEASIBLE(k))
  3  T_ref ← max_k T(Γ[k])
  4  E ← DETECT_CONFLICTS(Γ, P)                          # records (k, j, t_first, t_last, d_min)
  5  clusters ← CONNECTED_COMPONENTS(E);  mark all unresolved
  6  while ∃ unresolved cluster N:
  7     fixed ← {Γ[j] : j ∉ N};  round ← 0
  8     loop
  9        for k ∈ N:  A_k ← GENERATE_ALTERNATIVES(env, k, routes[k], E|k, B, T_ref, P)
 10        if ∃k: A_k = ∅:  mark N INFEASIBLE; break
 11        X, bg ← CONFLICT_TABLES(N, A, fixed, P)    # cached |A_k|×|A_j| boolean tables
 12        a ← (a_k^0)_{k∈N}                           # nominal; any start works for termination
 13        (a, converged, it) ← BEST_RESPONSE(N, A, X, bg, a, P.iter_cap)
 14        if C(a) > 0 and Π_k|A_k| ≤ P.exact_limit:
 15            a ← EXACT_SOLVE(N, A, X, bg);  converged ← VERIFY_NASH(N, A, X, bg, a)
 16        if C(a) = 0: break
 17        if round < P.aug_rounds: enlarge Δ_k / R_k for players still in conflict; round += 1
 18        else: mark N INFEASIBLE with residual conflicts; break
 19     if N not INFEASIBLE:
 20        for k ∈ N: Γ[k] ← Γ_k^{a_k}
 21        E_new ← DETECT_CONFLICTS(N vs N^c)
 22        if E_new ≠ ∅: merge N with every cluster it now touches → unresolved   # ≤ K−1 merges
 23        else: mark N RESOLVED (store a, converged, it)
 24  V ← VALIDATE_JOINT(env, Γ, B, P)                   # all K MSs, independent of the game
 25  status ← SUCCESS if V.ok and no INFEASIBLE cluster else FAILURE(residuals, V)
 26  return (Γ, report, status)
```

7.2 Best-response dynamics and the cost

```
procedure BEST_RESPONSE(N, A, X, bg, a, cap)
  it ← 0;  seen ← {hash(a)}
  repeat
     improved ← false
     for k in ORDER(N):                    # deterministic: most conflicts first, then id
        b* ← lex-argmin_{b∈A_k} J_k(b, a_{−k})   # ties → keep a_k, then lowest index
        if J_k(b*, a_{−k}) <_lex J_k(a) − ε:
           a_k ← b*;  improved ← true;  it += 1
           assert hash(a) ∉ seen;  seen.add(hash(a))   # violation ⇒ bug in A1–A6
           if it ≥ cap: return (a, false, it)
  until not improved
  return (a, VERIFY_NASH(N, A, X, bg, a), it)

J_k(a) = ( Σ_{j∈N, j≠k} X[k,j][a_k,a_j] + bg[k][a_k],  E_k(a_k),  ΔL_k(a_k),  W_k(a_k) )
```

7.3 Alternatives, conflict detection and helpers

```
procedure GENERATE_ALTERNATIVES(env, k, route, E_k, B, T_ref, P)
  t_c ← earliest conflict time of k;  H ← {last stationary node before t_c (sensor or BS)}
  R ← {r0} ∪ DETOURS(leg containing t_c) ∪ LANES (only if free width ≥ d_sep)
  for r ∈ R, h ∈ H(r), δ ∈ GRID(Δ_h, B − T(r)) ∪ TARGETED(E_k):
      Γ ← BUILD_TIMED_TRAJ(env, route, r, {h: δ}, Δt)
      if VALIDATE_INDIVIDUAL(env, Γ, B): add (r, h, δ, Γ, E, ΔL, W)
  return PRUNE(A, A_max)           # Pareto in (E, ΔL, W) + timing diversity; keep r0

procedure DETECT_CONFLICTS(Γ, P)   # only pairs whose time windows and bounding boxes overlap
  for each such pair: d ← min_n ‖p_k(t_n) − p_j(t_n)‖ (non-docked times only)
      if d < d_sep + vΔt + ε_geo: record (k, j, t_first, t_last, d)

procedure EXACT_SOLVE(N, A, X, bg)      # DFS, players sorted by |A_k|; prune when partial Φ ≥_lex incumbent
procedure VERIFY_NASH(...)              # for every k and every b ∈ A_k: no strict lexicographic improvement
procedure VALIDATE_JOINT(env, Γ, B, P)  # V1–V4 per MS + all-pairs conservative check;
                                        # returns min separation, min clearance, max T_k
```

7.4 When no feasible alternative exists

NOMINAL_INFEASIBLE or FAILURE goes back to `main.py`, which escalates step by step, each step bounded:

1. enlarge the alternative sets (up to aug_rounds);
2. re-optimise the visiting order of the affected tours;
3. re-partition, or use $K + 1 \le K_{max}$;
4. otherwise, declare the instance infeasible under the model.

K is priority 1, so changing it is a global decision, never a local one.

7.5 Guarantees

Guarantee	Holds when	Conditions
Collision-free	Only if status = SUCCESS	Certified by VALIDATE_JOINT via the sampling lemma. Assumes speed ≤ v, open-loop execution with exact timing, $d_{sep} = 2\rho +$ margin, and the declared BS dock exemption.
Budget-feasible	Every returned trajectory	A_k filtering plus the joint re-check, with T computed from the actual trajectory.
Finite termination	Always	Best response is finite (FIP) and capped; exact search is finite; augmentation rounds are bounded; at most $K - 1$ merges.
Nash equilibrium	Best response converges under the cap, or exact search completes	An NE of the game restricted to the generated alternatives, with non-cluster MSs fixed. Certified by VERIFY_NASH. **Not** an NE over all continuous trajectories.

8. Implementation architecture

> **In short**
>
> Add a small `trajectory.py` that builds exact, uniformly timed trajectories. Rewrite `corridor_game.py` around the functions below, keeping the old code only as a baseline. Hook it into `main.py` at Steps 8–11, and let the visualizer only draw results.

8.1 Data structures (corridor_game.py and trajectory.py)

```
@dataclass(frozen=True)
class TimedTrajectory:
    t: np.ndarray        # uniform grid, t[n] = n*dt
    xy: np.ndarray       # (n,2), exact positions at t[n]
    heading: np.ndarray  # (n,)
    dt: float; T: float; L: float; hold: float
    depart_idx: int; arrive_idx: int      # dock exemption window

@dataclass(frozen=True)
class Alternative:
    k: int; idx: int; variant: str        # "nominal" | "hold@S17" | "detour@leg3" | ...
    hold_node: int | None; hold_time: float
    traj: TimedTrajectory
    E: float; dL: float; W: float         # own-action cost components

@dataclass
class ConflictRecord:  k: int; j: int; t_first: float; t_last: float; d_min: float

@dataclass
class ClusterResult:
    players: list[int]; profile: dict[int, int]; converged: bool; nash_verified: bool
    iterations: int; potential_trace: list[tuple]; residual: list[ConflictRecord]; status: str

@dataclass
class ResolutionReport:
    clusters: list[ClusterResult]; merges: int; runtime: float
    min_separation: float; min_clearance: float; max_T: float; status: str
```

8.2 Functions

Function	Input → output	Responsibility
`build_timed_trajectory(env, route, variant, holds, dt, G, V)`	→ TimedTrajectory	Global arc-length timing (fixes Probe 4); exact dwell and hold; headings
`validate_individual(env, traj, B)`	→ (ok, reasons)	Checks V1–V4
`detect_conflicts(trajs, env)`	→ list[ConflictRecord]	Time-window / bounding-box pruning, then vectorised conservative check
`conflict_clusters(records)`	→ list[set[int]]	Connected components (networkx)
`generate_alternatives(env, k, route, recs, B, T_ref, G, V, P)`	→ list[Alternative]	Holds, detours, lanes, pruning
`conflict_tables(players, alts, fixed, P)`	→ X, bg	Cached symmetric boolean tables
`player_cost(k, profile, X, bg, alts)`	→ tuple	J_k
`best_response_dynamics(...)`	→ ClusterResult	Sequential, strict, capped
`verify_nash(...)`	→ bool	Explicit NE certificate
`exact_cluster_solve(...)`	→ profile	DFS with branch-and-bound on Φ
`validate_joint(env, trajs, B, P)`	→ report fields	Independent final check
`resolve_conflicts(env, routes, P)`	→ (trajs, report)	Top level, called from main.py
`legacy_go_yield_resolve`	current code, unchanged	Kept only as baseline M1

8.3 Interfaces with existing modules

Changes are limited to what a correct interface needs.

- **dubins.py:** add `generate_dubins_path_checked(q0, q1, R) -> (points, headings, length, ok)`, so the straight-line fall-back (*dubins.py, lines 240–241*) is reported instead of silently accepted. Existing callers are unaffected.
- **environment.py:** add vehicle length ℓ, width w, δ_{margin}, the derived ρ and d_{sep}, r_{dock} and hold_quantum. Document what `d_safe` means: today it is used both as obstacle clearance and (separately, as a hard-coded 3.0) for vehicle separation.
- **visualize.py:** move trajectory building into `trajectory.py`. The visualizer then only **renders** the output of `resolve_conflicts`. Remove the call at *visualize.py, lines 224–225*: visualization should not modify plans.
- **main.py:**
  - replace the no-op Steps 8–11 (*main.py, lines 78–81*) with `resolve_conflicts(env, routes, P)`;
  - pass B = `env.budget`;
  - check the budget with `TimedTrajectory.T`, not the Euclidean `compute_T`;
  - implement the escalation ladder on FAILURE.
- **Corridors:** an optional helper `find_corridors(env)` returns obstacle pairs whose gap is below a threshold (using shapely `nearest_points`). It is used only to generate detours and lanes and for reporting. **Correctness does not depend on corridors**, because conflicts are detected between whole trajectories; several corridors are therefore handled automatically by the clusters.
- **To make corridor scenarios exist at all:** the obstacle generator must allow gaps between 2(ρ + δ) and 2(R + clearance), and the planner must inflate by ρ + δ and check Dubins arcs explicitly (V2) instead of inflating by 2R.

8.4 Avoiding infinite loops

- strict improvement plus FIP, with an iteration cap;
- exact search is finite and has a size limit;
- augmentation rounds are bounded;
- cluster merges strictly reduce the number of clusters;
- an overall wall-clock limit.

8.5 Complexity

- Alternatives: $O(\Sigma_k |A_k| \cdot C_{plan})$.
- Conflict tables: $O(\Sigma_{k<j} |A_k||A_j| \cdot S)$, where S is the number of overlapping time samples (before pruning).
- Each best-response step: $O(|A_k| \cdot |N|)$ table look-ups.
- Exact search: $O(\Pi |A_k|)$ worst case, used only on small clusters.

8.6 Unit tests (pytest)

1. **Potential identity:** on random symmetric games, $\Delta J_k = \Delta\Phi$ for every deviation (as in Check [1]).
2. **Termination:** best response on random games ends at a profile that passes `verify_nash`.
3. **Asymmetry guard:** an asymmetric χ makes test 1 fail (documents assumption A5).
4. **NE with conflict:** the Check [3] game. Best response stops with a conflict; `exact_cluster_solve` finds (1,1).

5. **Simultaneous vs sequential:** the Check [4] game cycles under simultaneous updates and terminates under sequential ones.
6. **Sampling lemma:** the Probe 2 geometry is detected by `detect_conflicts` and missed by the legacy index check.
7. **Time grid:** `build_timed_trajectory` gives $t[-1] = L/v + \Sigma\tau + \delta$ within dt, and every step is $\le v \cdot dt$.
8. **Head-on single-lane corridor:** one MS holds at its upstream sensor; joint validation reports minimum separation $\ge d_{sep}$.
9. **Three in a row:** no conflicting pair is left both moving (regression for Probe 3b).
10. **Budget filter:** no alternative with $T > B$ ever appears in A_k.
11. **Dubins validation:** a straight-line fall-back and a heading discontinuity are both rejected.
12. **Obstacle sweep:** an alternative that clips an obstacle is rejected.
13. **Infeasible instance:** returns FAILURE with residual conflicts — never a silent success.
14. **Determinism:** the same inputs give an identical output and report.
15. **BS dock and departure:** staggered departures, or a dock exemption limited to r_{dock}.

9. Testing and evaluation plan

> **In short**
>
> Compare four methods on the same instances, scored by the same independent validator. Measure safety, budget, time, length, waiting and runtime, and run ablations that switch individual design choices off. State expected results as hypotheses and test them; do not claim superiority in advance.

9.1 Methods

ID	Method	Description
M1	Existing Go/Yield code	`legacy_go_yield_resolve`, unchanged.
M2	Approach 1, formalised	Rosenthal game over resources {lane L, lane R, detour route(s)} with BPR-type latency $\ell_e(n) = t_e(1 + \kappa(n/cap_e)^p)$ plus own detour time. Spatial only. Lanes offered only when free width $\ge d_{sep}$. Budget filtered with free-flow time.
M3	Recommended method	Sections 5–8.
M3-exact	Reference	Global minimisation of Φ — quality reference on small clusters.
M4	Prioritized planning	MSs ordered by increasing slack. Each takes the minimum-cost alternative from the **same** A_k that is conflict-free against the MSs already committed.

9.2 Instances

1. The random generator in `main.py`, seeded (open-space encounters).
2. A new corridor generator with controlled widths (single lane / double lane / U-turn capable) and lengths, with $K \in \{2, \ldots, 8\}$.
3. Small designed scenarios: head-on; following in the same direction; crossing at a corridor mouth; three MSs at one corridor; two coupled corridors; a deliberately infeasible case.
4. Budget tightness $(B - T_{max}^0)/B \in \{0.02, 0.05, 0.1, 0.2, 0.4\}$.

9.3 Fair conditions

- Nominal GA routes computed once per seed and shared by all methods.
- The same timed-trajectory builder, Δt, d_{sep} and ρ for all methods.
- **Every output scored by the same independent `validate_joint`** — never by a method's own detector.
- Identical alternative sets for M3 and M4; the same detour generator for M2.
- The same time limit and hardware.
- At least 100 seeds per configuration.
- Report median and IQR, success rates with Wilson confidence intervals, and paired tests (Wilcoxon signed-rank; McNemar for success/failure) with Holm correction.

9.4 Metrics

Metric	How it is measured
Conflicts detected	Pairs flagged by `detect_conflicts` on the nominal plan
Residual collisions	Pairs with $\chi = 1$ in the final plan (continuous-conservative check)
Minimum separation	Smallest centre-to-centre distance over all pairs and times
Budget violations	Number of MSs with $T_k > B$
Total waiting	$\Sigma_k W_k$ (s)
Extra length	$\Delta L_{total} = \Sigma_k \Delta L_k$ (m)
Makespan	T_{max}, and its change versus T_{ref}
Successful missions	Instances with status SUCCESS
Infeasible resolutions	Clusters marked INFEASIBLE
Best-response iterations	Improvement steps per cluster
Exact-search activations	Clusters where EXACT_SOLVE ran
Cluster merges	Number of merge events
Runtime	Wall-clock time of conflict resolution
Pure NE reached	M1: N/A (no game defined). M2: checked against its own congestion game. M3: `verify_nash` certificate. M4: N/A, but check after the fact whether M4's output happens to be an NE of M3's game.

9.5 Ablations

1. Hold-only alternatives (a reservation game).
2. Route-only alternatives (close to the PDF).
3. Lexicographic cost vs scalar weights.
4. Update order (round-robin / largest gain / random): termination must hold for all; quality may differ.
5. With and without the exact fallback.
6. Cluster games vs one game over the whole fleet.
7. Sensitivity to Δt and the margin (false-positive conflicts).
8. Size of A_k and the hold quantum Δ_h.
9. T_{ref} guard on/off (effect on T_{max}).
10. Background term on/off.

9.6 Hypotheses (to be tested, not assumed)

- **H1:** M3 has fewer residual collisions than M1.
- **H2:** M3 and M4 differ in T_{max} and ΔL_{total}.

10. Research limitations and claims

> **In short**
>
> The building blocks are standard and must be cited, not claimed. What may be new is how they are combined for multi-mobile-sink Dubins data collection. Avoid claims the method cannot support, and be ready for the reviewer question “why not just optimise Φ centrally?”.

10.1 Standard results being used (cite, do not claim)

- Potential games and the FIP (Monderer & Shapley 1996).
- Congestion games (Rosenthal 1973).
- Player-specific and weighted variants (Milchtaich 1996; Harks & Klimm 2012).
- PLS-hardness of pure NE (Fabrikant, Papadimitriou & Talwar 2004).
- Potential games for cooperative control (Marden, Arslan & Shamma 2009).
- Generalized Nash problems and generalized potential games (Facchinei & Kanzow 2010; Facchinei, Piccialli & Sciandrone 2011).
- Independence detection and cluster merging (Standley 2010).
- Conflict-based search, CBS (Sharon et al. 2015), and its continuous-time version, CCBS (Andreychuk et al. 2022).
- Prioritized planning (Erdmann & Lozano-Pérez 1987; Silver 2005).
- Conservative sampling margins.

10.2 Possibly novel (subject to a literature review)

- Applying this to multi-mobile-sink Dubins-TSP data collection, with holds placed at sensor dwells.
- The lexicographic cost aligned with (K, T_{max}, L_{total}) and the makespan guard.
- The bounded escalation ladder tied to the outer partitioning loop.

10.3 Claims that would be unjustified

- “The NE is collision-free.”
- “The NE is socially or globally optimal.”
- “Polynomial-time convergence.”
- “Decentralized” — unless it actually runs on board with only local information.
- “Deadlock-free in all cases” — only “conflict-free if a conflict-free profile exists among the generated alternatives and exact search completes”.
- “Left/Right is always feasible.”
- “It solves the continuous trajectory game.”

10.4 The honest caveat a reviewer will raise

The planner is centralized and offline. In that setting, best-response dynamics is a provably terminating local search on Φ, and Φ could be minimised directly (which is what EXACT_SOLVE does for small clusters). What the game-theoretic framing adds:

- costs for each MS that are aligned with a single system-level objective;
- a stability certificate: no MS gains by deviating on its own;
- an anytime procedure that could be distributed.

That is a modest but defensible contribution. It should not be presented as more.

10.5 Other limitations

- Open-loop timing: real execution needs time-tracking control or replanning.
- Completeness only relative to the discretised alternatives.
- The vΔt margin produces false-positive conflicts.
- Exponential worst case.
- Disc-footprint model.
- The current planner has a 2R inflation and straight-line fall-backs that must be fixed first; otherwise the validator will rightly reject nominal routes.

Appendix A. Probes and numerical checks

Two scripts were run while preparing this review: `probe_corridor_game.py` (experiments on the real code) and `check_potential.py` (checks of the proposed mathematics).

ID	What it tests	Result
Probe 1	Head-on encounter on one line, resolved by `game_theory_resolve`	Minimum distance after resolution 0.000 (one wait inserted)
Probe 2	Opposing pass, lateral offset 1.7, samples straddling the closest approach	Sampled minimum 3.023 → not detected; true minimum 1.701
Probe 3 / 3b	Three robots in a row; one frame of the decision loop	All yield needlessly, or the conflicting pair (0,1) both Go
Probe 4	Frame spacing of a real 3-sensor tour	17 non-uniform steps; frames imply 35.00 s vs true 32.78 s
Probe 5	Obstacle generator and planner inflation	Physical gaps > 24; narrow threshold 14; planner closes gaps < 24
Check [1]	Exact potential identity, 200 random games, every unilateral deviation	max $\lvert\Delta J_k - \Delta\Phi\rvert = 8.9\times10^{-16}$
Check [2]	Sequential strict best response, 500 random games	All terminated at a verified pure NE; steps mean 2.9, max 9
Check [3]	Two-player game with χ = [[1,1],[1,0]]	(0,0) is a pure NE with a conflict; (1,1) is conflict-free
Check [4]	Simultaneous best response in a “conflict if same choice” game	Cycles (0,0) → (1,1) → (0,0) → …
Check [5]	Conservative sampling lemma, 2,000 random speed-bounded motion pairs	0 violations

Appendix B. References

Verify the bibliographic details before citing.

1. R. W. Rosenthal (1973). A class of games possessing pure-strategy Nash equilibria. *International Journal of Game Theory* 2, 65–67.
2. D. Monderer and L. S. Shapley (1996). Potential games. *Games and Economic Behavior* 14, 124–143.
3. I. Milchtaich (1996). Congestion games with player-specific payoff functions. *Games and Economic Behavior* 13, 111–124.
4. T. Harks and M. Klimm (2012). On the existence of pure Nash equilibria in weighted congestion games. *Mathematics of Operations Research* 37(3), 419–436.
5. A. Fabrikant, C. Papadimitriou and K. Talwar (2004). The complexity of pure Nash equilibria. *Proc. ACM STOC*.
6. J. R. Marden, G. Arslan and J. S. Shamma (2009). Cooperative control and potential games. *IEEE Trans. Systems, Man, and Cybernetics, Part B* 39(6), 1393–1407.
7. F. Facchinei and C. Kanzow (2010). Generalized Nash equilibrium problems. *Annals of Operations Research* 175, 177–211.
8. F. Facchinei, V. Piccialli and M. Sciandrone (2011). Decomposition algorithms for generalized potential games. *Computational Optimization and Applications* 50, 237–262.
9. T. Standley (2010). Finding optimal solutions to cooperative pathfinding problems. *Proc. AAAI*.
10. G. Sharon, R. Stern, A. Felner and N. Sturtevant (2015). Conflict-based search for optimal multi-agent pathfinding. *Artificial Intelligence* 219, 40–66.
11. A. Andreychuk, K. Yakovlev, P. Surynek, D. Atzmon and R. Stern (2022). Multi-agent pathfinding with continuous time. *Artificial Intelligence* 305.
12. M. Erdmann and T. Lozano-Pérez (1987). On multiple moving objects. *Algorithmica* 2, 477–521.
13. D. Silver (2005). Cooperative pathfinding. *Proc. AIIDE*.